

系统开发工具基础第四周实验报告

姓名：刘静雅

学号：24160001055

日期：2026 年 9 月 14 日

目录

1	实验环境与目录结构	2
2	课上实验	2
2.1	第 13 题：建立可执行的本地质量门禁	2
2.2	第 14 题：让 Make 只重建真正受影响的产物	4
2.3	第 15 题：把本地 API 数据转换为可读报告	7
2.4	第 16 题：修复并交付一个陌生的小型工具仓库	9
3	课后练习	12
3.1	代码格式化、静态检查与 AI 修复	12
3.2	提交前自动检查	14
3.3	pytest 单元测试	14
3.4	覆盖率统计与 HTML 报告	15
3.5	人工补充测试与覆盖率提升	17
3.6	AI agent 辅助测试改进	17
3.7	GitHub Actions 持续集成	18
3.8	持续集成失败与恢复	19
3.9	正则检查与 Markdown 替换	21
3.10	JSON 字段提取与解析器	22
4	实验总结与感悟	23

1 实验环境与目录结构

本周实验在 WSL 的 Bash 环境中完成，仓库目录为 `~/sys-dev-tools-lab/week04`。实验主要使用 Python、pytest、ruff、Make、curl、jq、Git 等工具，完成代码质量检查、自动化构建、本地 API 数据处理与软件包交付等任务。

本周实验目录结构如下：

```
week04/  
|-- q13/  
|-- q14/  
|-- q15/  
|-- q16/  
|-- report_assets/  
|   |-- practice/  
|-- report.tex
```

2 课上实验

2.1 第 13 题：建立可执行的本地质量门禁

题目原文

复用第 10 题完成的 greetlab 包，在 q13 中建立一个轻量本地质量检查。

- 复制 q10 为 q13；在 `pyproject.toml` 中加入 ruff 和 pytest 的最小配置。
- 补充一个正常姓名测试和一个空白姓名测试，每个测试包含有效断言。
- 运行 `ruff format`、`ruff check` 和 `pytest`，修复全部问题，不得全局忽略规则。
- 编写 `check.sh`，依次执行 `ruff format --check`、`ruff check` 和 `pytest`；运行并保证返回 0。

实验步骤

首先复用前一周完成的 greetlab 项目，将 q10 复制为 q13。在 `pyproject.toml` 中补充 pytest 与 ruff 的最小配置，使测试框架能够定位 `src` 与 `tests` 目录，同时为代码格式检查和静态检查提供统一规则。

随后补充正常姓名和空白姓名两个测试用例。正常姓名测试验证程序能够输出正确问候语；空白姓名测试验证程序能够拒绝无效输入。完成代码修改后，分别运行 `pytest`、`ruff format --check .` 和 `ruff check`。

最后编写 `check.sh`，将格式检查、静态检查和单元测试串联为一个统一的本地质量门禁入口。脚本的核心形式如下：

```
#!/usr/bin/env bash
set -euo pipefail

ruff format --check .
ruff check .
pytest
```

实验结果与分析

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ sed -n '1,220p' pyproject.toml
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-24160001055"
version = "0.1.0"
requires-python = ">=3.10"

[project.scripts]
sdt-greet = "greetlab.cli:main"

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]

[tool.ruff]
target-version = "py312"
line-length = 88

[tool.ruff.lint]
select = ["E", "F", "I"]
```

图 1: 第 13 题 `pyproject.toml` 中的 `pytest` 与 `ruff` 配置

实验完成后，`pyproject.toml` 中已经配置了 `pytest` 的 `pythonpath` 与 `testpaths`，并设置了 `ruff` 的目标 Python 版本、行长度和检查规则。这些配置使质量检查可以在项目目录中稳定执行。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ pytest
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.01s =====
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ ruff format --check . && ruff check
4 files already formatted
All checks passed!
```

图 2: 第 13 题 `pytest`、`ruff format` 与 `ruff check` 通过

执行检查命令后，`pytest` 共收集两个测试项目，最终结果为“2 passed”。随后

```
ruff format --check .
```

显示全部文件格式正确，`ruff check` 显示 “All checks passed!”，说明测试、格式与静态检查均通过。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ nano check.sh
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ chmod +x check.sh
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/q13
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.00s =====
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q13$ echo $?
0
```

图 3: 第 13 题 `check.sh` 执行结果与退出码

执行 `check.sh` 后，格式检查、静态检查和测试均成功完成，最后 `echo $?` 输出为 0。返回值 0 表明该质量门禁脚本执行成功，可在提交前作为统一检查入口使用。

2.2 第 14 题：让 Make 只重建真正受影响的产物

题目原文

在 q14 中按给定内容创建三个源文件，只需完成 Makefile。

给定内容如下：

```
# data.csv
name,value
alpha,2
beta,3
gamma,5

# stats.py
import csv
with open("data.csv") as f:
    total = sum(int(r["value"]) for r in csv.DictReader(f))
open("stats.txt", "w").write(str(total))

# build_report.py
text = open("report.md").read()
total = open("stats.txt").read()
open("report.txt", "w").write(f"{text}\nTotal: {total}\n")
```

```
# report.md
# Data Report
```

- 创建给定的 data.csv、stats.py、build_report.py 和 report.md。
- 编写 Makefile，包含 all、stats.txt、report.txt 和 clean；准确列出数据文件、脚本和中间产物依赖，并把 all、clean 声明为.PHONY。
- 验证首次 make 生成两个产物；第二次无改动时不执行配方。
- touch data.csv 后再次 make，确认 stats.txt 和 report.txt 重新生成；clean 只删除生成物。

实验步骤

本题先创建数据文件 data.csv、统计脚本 stats.py、报告生成脚本 build_report.py 以及报告标题文件 report.md。其中，stats.py 用于计算 CSV 文件中数值列的总和，build_report.py 用于读取统计结果并生成最终报告。

随后编写 Makefile，明确目标文件之间的依赖关系。其核心内容如下：

```
.PHONY: all clean

all: report.txt

stats.txt: data.csv stats.py
    python3 stats.py

report.txt: stats.txt report.md build_report.py
    python3 build_report.py

clean:
    rm -f stats.txt report.txt
```

通过首次执行 make、再次执行 make、执行 touch data.csv 后重新执行 make，可以验证 Make 的增量构建机制。最后使用 make clean 删除构建产生的中间文件与结果文件。

实验结果与分析

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano data.csv
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano stats.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano build_report.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano report.md
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ nano Makefile
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make
python3 stats.py
python3 build_report.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ cat stats.txt
10
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ cat report.txt
# 第4周数据报告

Total: 10
```

图 4: 第 14 题首次 make 构建及生成结果

首次执行 make 时,终端依次运行 python3 stats.py 和 python3 build_report.py。生成的 stats.txt 内容为 10, report.txt 中包含报告标题和“Total: 10”,说明数据统计与报告生成均成功完成。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make
make: Nothing to be done for 'all'.
```

图 5: 第 14 题第二次 make 未重复构建

第二次直接运行 make 时,终端显示“Nothing to be done for ‘all.’”。这表示源文件没有发生变化,目标文件仍然是最新状态,因此 Make 没有重复执行构建命令。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ touch data.csv
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make
python3 stats.py
python3 build_report.py
```

图 6: 第 14 题更新 data.csv 后重新构建

执行 touch data.csv 后再次运行 make,终端重新执行 stats.py 与 build_report.py。这是因为数据文件的时间戳更新后,stats.txt 以及依赖它的 report.txt 都需要重新生成。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ make clean
rm -f stats.txt report.txt
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q14$ find . -maxdepth 1 -type f -printf '%f\n' | sort
Makefile
build_report.py
data.csv
report.md
stats.py
```

图 7: 第 14 题 make clean 清理结果

执行 `make clean` 后，终端删除 `stats.txt` 和 `report.txt`。随后列出的目录文件中仅保留 `Makefile`、源数据文件、`Python` 脚本和 `Markdown` 文件，说明 `clean` 目标只清理生成物，没有删除源文件。

2.3 第 15 题：把本地 API 数据转换为可读报告

题目原文

在 `q15` 中创建给定 `JSON` 和本地 `HTTP` 服务，再用 `curl` 与 `jq` 生成简短 `Markdown` 报告。

给定 `JSON` 如下：

```
[
{"name":"alpha","status":"active","downloads":120,"version":"1.2.0"},
{"name":"beta","status":"inactive","downloads":900,"version":"2.0.0"},
{"name":"gamma","status":"active","downloads":450,"version":"1.5.1"},
{"name":"delta","status":"active","downloads":450,"version":"0.9.0"},
{"name":"epsilon","status":"active","downloads":80,"version":"3.1.0"}
]
```

- 把给定 `JSON` 保存为 `packages.json`，并在 `q15` 目录运行 `python -m http.server 8000`。
- 使用 `curl -fsS` 获取 `http://127.0.0.1:8000/packages.json`。
- 使用 `jq` 筛选 `status` 为 `active` 且 `downloads` 不少于 100 的记录，并按 `downloads` 降序、`name` 升序排列。
- 编写 `api_report.sh`，把结果生成 `summary.md`，包含标题和 `name/version/downloads` 三列表格。

实验步骤

首先将题目给定的 `JSON` 数据保存为 `packages.json`，并在 `q15` 目录启动本地 `HTTP` 服务。随后使用 `curl -fsS` 获取本地服务器返回的 `JSON` 数据，再使用 `jq` 完成筛选、排序和格式化。

脚本 `api_report.sh` 先输出 `Markdown` 标题与表头，再对 `JSON` 数组执行筛选和排序，最终生成 `summary.md`。筛选逻辑的关键部分如下：

```
map(select(.status == "active" and .downloads >= 100))
| sort_by(-.downloads, .name)
```

```
| .[]  
| "| \(.name) | \(.version) | \(.downloads) |"
```

筛选后排除 inactive 的 beta 和 downloads 小于 100 的 epsilon。delta 和 gamma 的下载量同为 450，因此按照名称升序排列，最终顺序为 delta、gamma、alpha。

实验结果与分析

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ curl -fsS http://127.0.0.1:8000/packages.json | jq .  
[  
  {  
    "name": "alpha",  
    "status": "active",  
    "downloads": 120,  
    "version": "1.2.0"  
  },  
  {  
    "name": "beta",  
    "status": "inactive",  
    "downloads": 900,  
    "version": "2.0.0"  
  },  
  {  
    "name": "gamma",  
    "status": "active",  
    "downloads": 450,  
    "version": "1.5.1"  
  },  
  {  
    "name": "delta",  
    "status": "active",  
    "downloads": 450,  
    "version": "0.9.0"  
  },  
  {  
    "name": "epsilon",  
    "status": "active",  
    "downloads": 80,  
    "version": "3.1.0"  
  }  
]
```

图 8: 第 15 题 curl 获取本地 JSON 数据

执行 `curl -fsS http://127.0.0.1:8000/packages.json` 后，命令能够成功访问本地 HTTP 服务，jq 将 JSON 数据格式化输出。该结果说明本地 API 服务和数据文件均可正常访问。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ sed -n '1,220p' api_report.sh  
#!/usr/bin/env bash  
set -euo pipefail  
  
{  
  printf '# 本地 API 软件包报告\n\n'  
  printf '| name | version | downloads |\n'  
  printf '| --- | --- | ---: |\n'  
  
  curl -fsS http://127.0.0.1:8000/packages.json |  
    jq -r '  
    map(select(.status == "active" and .downloads >= 100))  
    | sort_by(-.downloads, .name)  
    | .[]  
    | "| \(.name) | \(.version) | \(.downloads) |"  
    ,  
  } > summary.md
```

图 9: 第 15 题 api_report.sh 脚本内容

脚本使用 `set -euo pipefail` 保证命令执行出现错误时立即停止。脚本通过 `curl` 获取 JSON，再使用 `jq` 筛选 `active` 状态且 `downloads` 不少于 100 的记录，并将结果写入 `summary.md`。

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ ./api_report.sh
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q15$ cat summary.md
# 本地 API 软件包报告

| name | version | downloads |
| --- | --- | ---: |
| delta | 0.9.0 | 450 |
| gamma | 1.5.1 | 450 |
| alpha | 1.2.0 | 120 |
```

图 10: 第 15 题 `summary.md` 报告结果

生成的 Markdown 报告包含 `name`、`version`、`downloads` 三列，最终保留 `delta`、`gamma` 和 `alpha` 三条记录。结果中的排序符合下载量降序、名称升序的要求。

2.4 第 16 题：修复并交付一个陌生的小型工具仓库

题目原文

复用第 13 题的项目，在 q16 中完成一次 15 分钟综合检查。

- 复制 q13 为 q16 并初始化/继续使用 Git。
- 把问候语实现临时改成始终返回字面量 `Hello, name!`，运行 `check.sh` 确认测试失败；随后定位并修复。
- 编写最小 Makefile，包含 `check`、`build` 和 `clean`；`check` 调用 `check.sh`，`build` 生成 `wheel`。
- 运行 `make check` 和 `make build`，计算 `wheel` 的 SHA-256，并把最终改动提交为一个内容聚焦的 Git 提交。

实验步骤

本题首先将 q13 复制为 q16，并继续使用仓库根目录已有的 Git 管理。随后按题目要求，将问候语实现临时改为固定输出 `Hello, name!`，运行 `check.sh` 以复现测试失败。

在确认失败后，将输出逻辑修复为根据参数中的姓名输出问候语：

```
print(f"Hello, {name}!")
```

修复完成后重新执行 `check.sh`，确认代码质量检查和两个测试用例均通过。

之后编写最小 Makefile，将检查、构建与清理任务封装为 `make` 目标：

```
.PHONY: check build clean

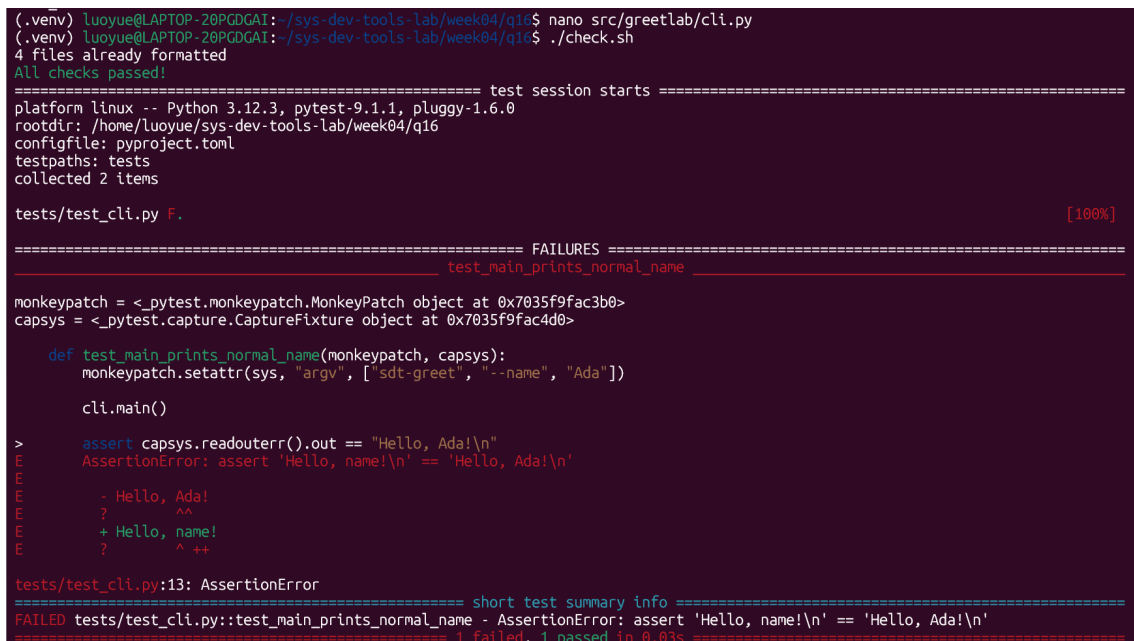
check:
    ./check.sh

build:
    python -m build

clean:
    rm -rf build dist src/*.egg-info
```

最后运行 `make build` 构建源码包和 wheel 包，并通过 `sha256sum dist/*.whl` 计算 wheel 文件的 SHA-256 校验值。

实验结果与分析



```
(.venv) luoyue@LAPTOP-20PCDGA1:~/sys-dev-tools-lab/week04/q16$ nano src/greetlab/cli.py
(.venv) luoyue@LAPTOP-20PCDGA1:~/sys-dev-tools-lab/week04/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/Luoyue/sys-dev-tools-lab/week04/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py F. [100%]

===== FAILURES =====
test_main_prints_normal_name

monkeypatch = <pytest.monkeypatch.MonkeyPatch object at 0x7035f9fac3b0>
capsys = <pytest.capture.CaptureFixture object at 0x7035f9fac4d0>

    def test_main_prints_normal_name(monkeypatch, capsys):
        monkeypatch.setattr(sys, "argv", ["sdt-greet", "--name", "Ada"])
        cli.main()

> assert capsys.readouterr().out == "Hello, Ada!\n"
AssertionError: assert 'Hello, name!\n' == 'Hello, Ada!\n'
- Hello, Ada!
?      ^^
+ Hello, name!
?      ^++

tests/test_cli.py:13: AssertionError
===== short test summary info =====
FAILED tests/test_cli.py::test_main_prints_normal_name - AssertionError: assert 'Hello, name!\n' == 'Hello, Ada!\n'
1 failed, 1 passed in 0.03s
```

图 11: 第 16 题故意修改问候语后测试失败

将问候语临时修改为固定文本 `Hello, name!` 后，重新运行测试，正常姓名测试失败。测试期望输出为 `Hello, Ada!`，但实际输出为 `Hello, name!`，因此触发断言错误。该结果说明已有测试能够有效发现功能回归。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ ./check.sh
4 files already formatted
All checks passed!
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/q16
configfile: pyproject.toml
testpaths: tests
collected 2 items

tests/test_cli.py .. [100%]

===== 2 passed in 0.00s =====
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ echo "exit_code=$?"
exit_code=0
```

图 12: 第 16 题修复后 check.sh 通过

恢复为基于姓名变量输出问候语后，执行 `check.sh` 的结果显示两个测试全部通过。终端输出 `exit_code=0`，说明格式检查、静态检查和测试均执行成功。

```
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ cat Makefile
.PHONY: check build clean

check:
    ./check.sh

build:
    python -m build

clean:
    rm -rf build dist src/*.egg-info
```

图 13: 第 16 题 Makefile 内容

Makefile 中定义了 `check`、`build` 和 `clean` 三个目标。其中 `check` 调用 `check.sh`，`build` 调用 `python -m build`，`clean` 用于删除构建产生的 `build`、`dist` 和 `egg-info` 文件夹。

```
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.linux-x86_64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_24160001055.egg-info to build/bdist.linux-x86_64/wheel/./greetlab_24160001055-0.1.0-py3.12.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_24160001055-0.1.0.dist-info/WHEEL
creating '/home/luoyue/sys-dev-tools-lab/week04/q16/dist/.tmp-xj5pme2j/greetlab_24160001055-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_24160001055-0.1.0.dist-info/METADATA'
adding 'greetlab_24160001055-0.1.0.dist-info/WHEEL'
adding 'greetlab_24160001055-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_24160001055-0.1.0.dist-info/top_level.txt'
adding 'greetlab_24160001055-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_24160001055-0.1.0.tar.gz and greetlab_24160001055-0.1.0-py3-none-any.whl
(.venv) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/q16$ sha256sum dist/*.whl
c7b76ade6741dba0694ede24d558d5e6cc6dd89ccef73c16f142701318446f8a dist/greetlab_24160001055-0.1.0-py3-none-any.whl
```

图 14: 第 16 题 make build 构建过程

执行 `make build` 后，命令成功调用 `python -m build`，并完成源码包构建流程。构建过程中出现 `README` 文件缺失的警告，但没有导致构建中断，后续仍成功生成了发布产物。

```
(.venv) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/q1e$ make build
python -m build
* Creating isolated environment: venv+pip...
* Installing packages in isolated environment:
  - setuptools>=68
* Getting build dependencies for sdist...
running egg_info
creating src/greetlab_24160001055.egg-info
writing src/greetlab_24160001055.egg-info/PKG-INFO
writing dependency links to src/greetlab_24160001055.egg-info/dependency_links.txt
writing entry points to src/greetlab_24160001055.egg-info/entry_points.txt
writing top-level names to src/greetlab_24160001055.egg-info/top_level.txt
writing manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
reading manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
* Installed build dependency versions:
  - setuptools==84.0.0
* Building sdist...
running sdist
running egg_info
writing src/greetlab_24160001055.egg-info/PKG-INFO
writing dependency links to src/greetlab_24160001055.egg-info/dependency_links.txt
writing entry points to src/greetlab_24160001055.egg-info/entry_points.txt
writing top-level names to src/greetlab_24160001055.egg-info/top_level.txt
reading manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
writing manifest file 'src/greetlab_24160001055.egg-info/SOURCES.txt'
warning: sdist: standard file not found: should have one of README, README.rst, README.txt, README.md

running check
creating greetlab_24160001055-0.1.0
creating greetlab_24160001055-0.1.0/src/greetlab
creating greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
creating greetlab_24160001055-0.1.0/tests
copying files to greetlab_24160001055-0.1.0...
copying pyproject.toml -> greetlab_24160001055-0.1.0
copying src/greetlab/_init_.py -> greetlab_24160001055-0.1.0/src/greetlab
copying src/greetlab/cli.py -> greetlab_24160001055-0.1.0/src/greetlab
copying src/greetlab_24160001055.egg-info/PKG-INFO -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/SOURCES.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/dependency_links.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/entry_points.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
copying src/greetlab_24160001055.egg-info/top_level.txt -> greetlab_24160001055-0.1.0/src/greetlab_24160001055.egg-info
```

图 15: 第 16 题 wheel 构建完成并计算 SHA-256

构建过程最终成功生成 `.tar.gz` 与 `.whl` 文件。随后使用 `sha256sum dist/*.whl` 输出 wheel 文件的 SHA-256 校验值。该值可用于验证构建产物的完整性，便于后续提交与交付时进行核验。

3 课后练习

本部分按照实际完成的十个练习记录实验过程。每题先列出题目要求，再说明实验方法，最后结合运行结果进行分析。练习均在独立的质量实践项目中完成，相关配置位于仓库的 `week04/practice/quality-lab` 目录。

3.1 代码格式化、静态检查与 AI 修复

题目原文

为正在进行的项目配置格式化器、linter 和 AI agent。自动格式化应当能够处理格式错误；对于 linter 错误，要求 AI agent 运行 linter 并根据结果迭代修复。修复完成后需要人工检查改动，确认代码功能没有被破坏。

实验步骤与分析

在质量实践项目中安装 Ruff，先执行以下检查命令并记录原始结果：

```
ruff format --check .
ruff check .
```

随后把检查信息和修复目标交给 AI agent，要求它运行相同的检查命令，只修改必要文件。修复结束后再次执行检查，并用 `git diff` 人工核对改动范围。自动格式化适合处理缩进、换行等机械性问题，而 linter 提示仍需结合代码语义确认。

实验结果与解释

```
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ nano src/greetlab/name_rules.py
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ cp src/greetlab/name_rules.py /tmp/name_
rules_before.py
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ ruff format .
1 file reformatted, 4 files left unchanged
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ sed -n '1,100p' src/greetlab/name_rules.
py
import os

def normalize_name(name):
    cleaned = name.strip()
    if not cleaned:
        raise ValueError("name must not be blank")
    if any(character.isdigit() for character in cleaned):
        raise ValueError("name must not contain digits")
    return " ".join(cleaned.split())
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ ruff check .
F401 [*] `os` imported but unused
--> src/greetlab/name_rules.py:1:8

1 | import os
  |         ^^
help: Remove unused import: `os`

- import os
1 |

Found 1 error.
[*] 1 fixable with the `--fix` option.
```

图 16: 课后练习一：AI 修复前的 Ruff 检查

```
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ diff -u /tmp/name_rules_before.py src/gr
eetlab/name_rules.py
--- /tmp/name_rules_before.py    2026-09-15 14:36:14.146353895 +0800
+++ src/greetlab/name_rules.py   2026-09-15 15:50:06.766478483 +0800
@@ -1,10 +1,7 @@
-import os
-
-
- def normalize_name(name):
-     cleaned= name.strip()
-     if not cleaned:
-         raise ValueError("name must not be blank")
-     if any(character.isdigit() for character in cleaned):
-         raise ValueError("name must not contain digits")
-     return " ".join(cleaned.split())
-
+
+ cleaned = name.strip()
+ if not cleaned:
+     raise ValueError("name must not be blank")
+ if any(character.isdigit() for character in cleaned):
+     raise ValueError("name must not contain digits")
+ return " ".join(cleaned.split())
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ ruff format --check .
5 files already formatted
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ ruff check .
All checks passed!
```

图 17: 课后练习一：AI 修复后的检查与人工复核

初始检查能够定位格式和静态检查问题；修复后再次运行 Ruff，检查通过，差异内容集中在格式调整和 linter 指出的代码位置。这说明 AI agent 能够参与修复循环，但最终结果仍需人工复核。

3.2 提交前自动检查

题目原文

为项目配置 pre-commit 钩子，使提交前自动运行格式化检查、linter 和测试。

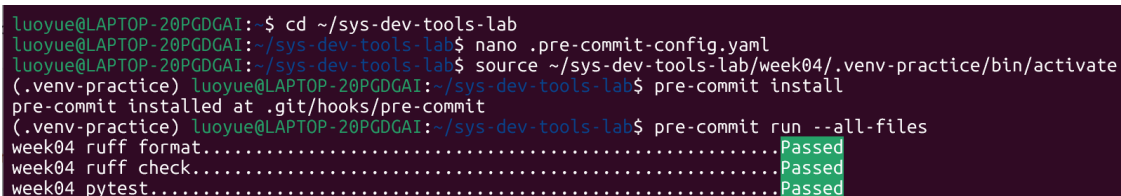
实验步骤与分析

在项目根目录创建 `.pre-commit-config.yaml`，将格式检查、静态检查和测试配置为本地钩子。安装 Git 钩子并检查全部文件：

```
pre-commit install
pre-commit run --all-files
```

这样可以把原本需要手动执行的质量检查接入提交流程。

实验结果与解释



```
luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ nano .pre-commit-config.yaml
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ source ~/sys-dev-tools-lab/week04/.venv-practice/bin/activate
(.venv-practice) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ pre-commit install
pre-commit installed at .git/hooks/pre-commit
(.venv-practice) luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab$ pre-commit run --all-files
week04 ruff format.....Passed
week04 ruff check.....Passed
week04 pytest.....Passed
```

图 18: 课后练习二：pre-commit 全部检查通过

全量检查完成后，格式检查、静态检查和测试均通过。结果表明，pre-commit 能够在提交前统一执行质量检查，减少遗漏手工检查的可能。

3.3 pytest 单元测试

题目原文

学习一种熟悉语言的测试库，并为正在进行的项目编写一个单元测试，运行测试并根据结果判断程序行为是否符合预期。

实验步骤与分析

在现有 Python 项目中补充 pytest 测试，分别覆盖正常输入、边界输入和无效输入，并通过断言验证函数返回值或命令行输出。执行 `pytest -v`，确认测试文件被正确收集，并根据失败信息修正实现或测试条件。

实验结果与解释

```
luoyue@LAPTOP-20PGDGAI: $ cd ~/sys-dev-tools-lab/week04/practice/quality-lab
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week04/practice/quality-lab$ mkdir -p tests
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week04/practice/quality-lab$ nano tests/test_name_rules.py
luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week04/practice/quality-lab$ pytest tests/test_name_rules.py -v
===== test session starts =====
platform linux -- Python 3.12.3, pytest-7.4.4, pluggy-1.4.0 -- /usr/bin/python3
cachedir: .pytest_cache
rootdir: /home/luoyue/sys-dev-tools-lab/week04/practice/quality-lab
configfile: pyproject.toml
collected 1 item

tests/test_name_rules.py::test_normalize_name_returns_normal_name PASSED [100%]

===== 1 passed in 0.01s =====
```

图 19: 课后练习三: pytest 单元测试结果

测试命令返回成功，说明测试文件被正确识别，断言全部成立。正常输入和异常输入分别验证了主要功能路径与错误处理路径。

3.4 覆盖率统计与 HTML 报告

题目原文

运行代码覆盖率工具并生成 HTML 格式报告，观察哪些代码行已经被测试覆盖，找出尚未覆盖的代码位置。

实验步骤与分析

使用以下命令同时生成终端统计与 HTML 报告：

```
pytest --cov=greetlab \
  --cov-report=term-missing \
  --cov-report=html
```

先查看终端中的覆盖率和缺失行，再打开 HTML 报告定位具体文件和代码行，为后续补充测试提供依据。

实验结果与解释

```
(.venv-practice) luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ python -m pytest \
> --cov=greetlab \
> --cov-report=term-missing \
> --cov-report=html
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/practice/quality-lab
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0, anyio-4.15.1
collected 4 items

tests/test_cli.py .. [ 50%]
tests/test_name_rules.py .. [100%]

===== tests coverage =====
coverage: platform linux, python 3.12.3-final-0
Name                               Stmts  Miss  Cover   Missing
-----
src/greetlab/__init__.py            0      0  100%
src/greetlab/cli.py                  9      0  100%
src/greetlab/name_rules.py          7      1   86%    6
TOTAL                               16      1   94%
Coverage HTML written to dir htmlcov
===== 4 passed in 0.04s =====
```

图 20: 课后练习四：初始覆盖率统计

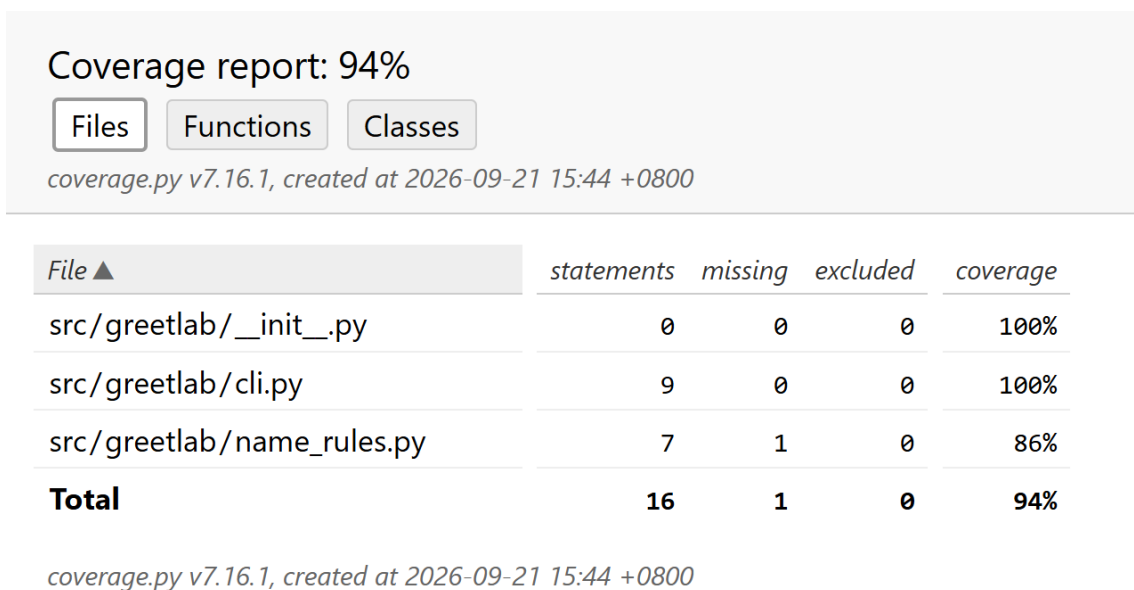


图 21: 课后练习四：HTML 覆盖率报告

初始覆盖率为 94%，HTML 报告标出了尚未执行的代码行。覆盖率不能直接证明程序没有错误，但能够明确指出测试没有涉及的路径。

3.5 人工补充测试与覆盖率提升

题目原文

根据覆盖率报告中未覆盖的代码行，手动编写测试以提高覆盖率，并重新运行测试确认新增测试有效。

实验步骤与分析

根据 HTML 报告定位未覆盖分支，分析这些分支对应的输入条件，然后在测试文件中补充针对性的边界测试或异常测试。重新运行带覆盖率的 pytest，并检查新增测试是否包含有效断言，而不是只为了执行代码。

实验结果与解释

```
(.venv-practice) luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week04/practice/quality-lab$ nano tests/test_name_rules.py
(.venv-practice) luoyue@LAPTOP-20PGDGAI: ~/sys-dev-tools-lab/week04/practice/quality-lab$ pytest \
> --cov=greetlab \
> --cov-report=term-missing \
> --cov-report=html
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/practice/quality-lab
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0, anyio-4.15.1
collected 4 items

tests/test_cli.py .. [ 50%]
tests/test_name_rules.py .. [100%]

===== tests coverage =====
coverage: platform linux, python 3.12.3-final-0
Name                               Stmts  Miss  Cover   Missing
-----
src/greetlab/__init__.py             0      0  100%
src/greetlab/cli.py                  9      0  100%
src/greetlab/name_rules.py           7      1   86%    6
-----
TOTAL                                16      1   94%
Coverage HTML written to dir htmlcov
===== 4 passed in 0.04s =====
```

图 22: 课后练习五：补充测试后的覆盖率结果

补充测试后，原先未执行的分支被实际运行，测试仍然全部通过，覆盖率得到提高。这说明提高覆盖率的关键是覆盖新的行为路径，而不是单纯增加测试数量。

3.6 AI agent 辅助测试改进

题目原文

使用 AI agent 提高项目覆盖率。要求 agent 以带覆盖率的方式运行测试，生成逐行覆盖率结果，并根据结果补充测试；同时人工判断 AI 生成的测试是否合理。

实验步骤与分析

将带有逐行覆盖率的测试结果提供给 AI agent，要求它先运行以下命令，再根据缺失行提出并实现测试：

```
pytest --cov=greetlab \
--cov-report=term-missing
```

生成测试后重新运行覆盖率命令，并检查测试是否验证真实行为，是否包含清晰断言。

实验结果与解释

```
luoyue@LAPTOP-20PGDCAI:~/sys-dev-tools-lab/week04/practice/quality-lab$ ./venv/bin/python -m pytest --cov=greetlab --cov-report=term-missing
===== test session starts =====
platform linux -- Python 3.12.3, pytest-9.1.1, pluggy-1.6.0
rootdir: /home/luoyue/sys-dev-tools-lab/week04/practice/quality-lab
configfile: pyproject.toml
testpaths: tests
plugins: cov-7.1.0
collected 5 items

tests/test_cli.py .. [ 40%]
tests/test_name_rules.py ... [100%]

===== tests coverage =====
coverage: platform linux, python 3.12.3-final-0

Name                                Stmts  Miss  Cover   Missing
-----
src/greetlab/__init__.py              0      0  100%
src/greetlab/cli.py                   9      0  100%
src/greetlab/name_rules.py           7      0  100%
-----
TOTAL                                16      0  100%

===== 5 passed in 0.03s =====
```

图 23: 课后练习六：AI agent 辅助补充测试后的覆盖率

AI agent 根据覆盖率结果补充了测试，重新运行后测试通过且覆盖率得到改善。人工检查确认新增测试具有实际输入和断言，说明 AI 可以提高编写效率，但不能替代测试设计与复核。

3.7 GitHub Actions 持续集成

题目原文

为项目设置持续集成，使其在每次 push 时运行格式化检查、lint 和测试。

实验步骤与分析

在 `.github/workflows/week04-quality.yml` 中配置 push 触发的工作流，指定 Python 版本，安装项目依赖，并依次执行格式检查、Ruff 检查和带覆盖率的 pytest。提交并推送后，在 GitHub Actions 页面查看运行状态。

实验结果与解释

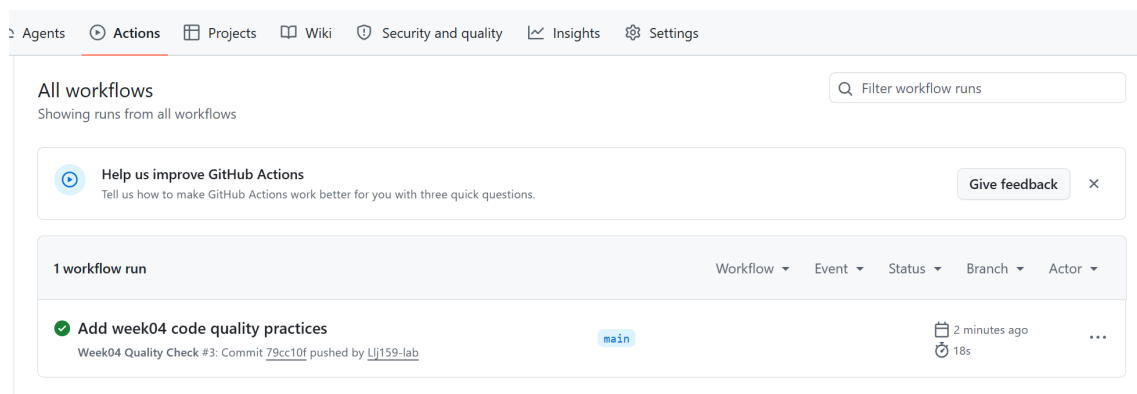


图 24: 课后练习七: GitHub Actions 检查通过

推送后, workflows 完成环境准备、依赖安装、格式检查、lint 和测试, 最终运行成功。这表明项目在干净的远程环境中也能通过质量检查。

3.8 持续集成失败与恢复

题目原文

故意破坏代码, 例如引入一条 linter 违规, 确认持续集成能够捕获问题; 随后修复错误并再次推送, 确认 CI 恢复成功。

实验步骤与分析

先在代码中加入一条可检测的 lint 违规, 提交并推送, 观察 CI 是否在质量检查阶段失败。随后撤销该错误, 再次运行本地检查并推送修复结果, 比较两次 workflow 状态。

实验结果与解释

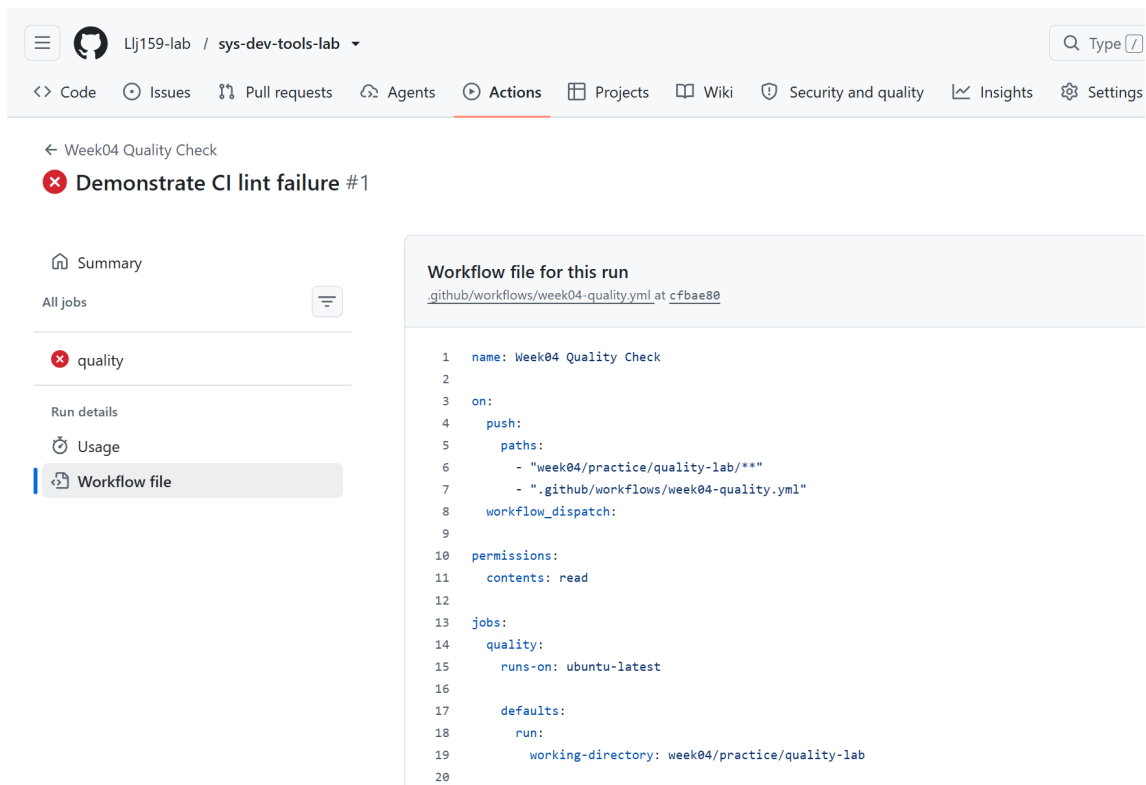


图 25: 课后练习八：故意引入问题后的 CI 失败

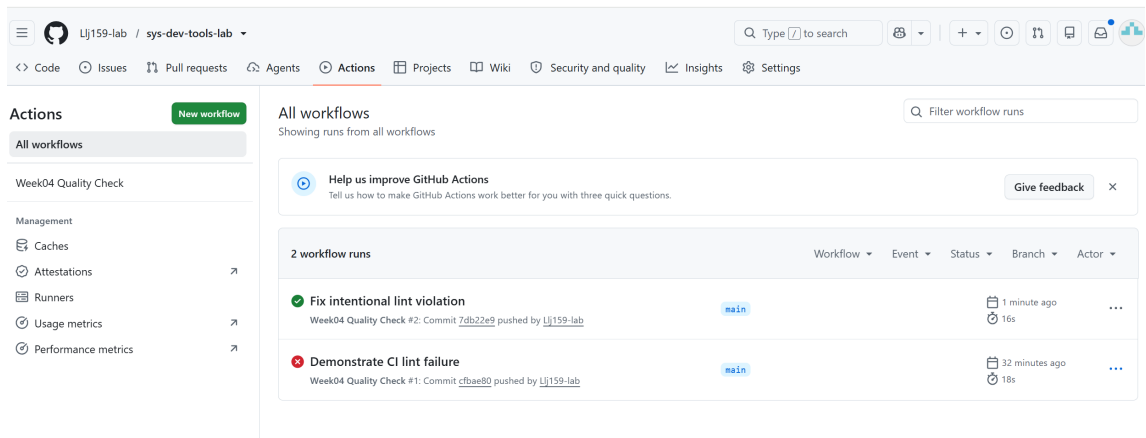


图 26: 课后练习八：修复问题后的 CI 恢复

引入 lint 违规后，CI 检查失败；修复并重新推送后，工作流恢复通过。前后结果证明工作流实际执行了质量检查，而不是只创建了配置文件。

3.9 正则检查与 Markdown 替换

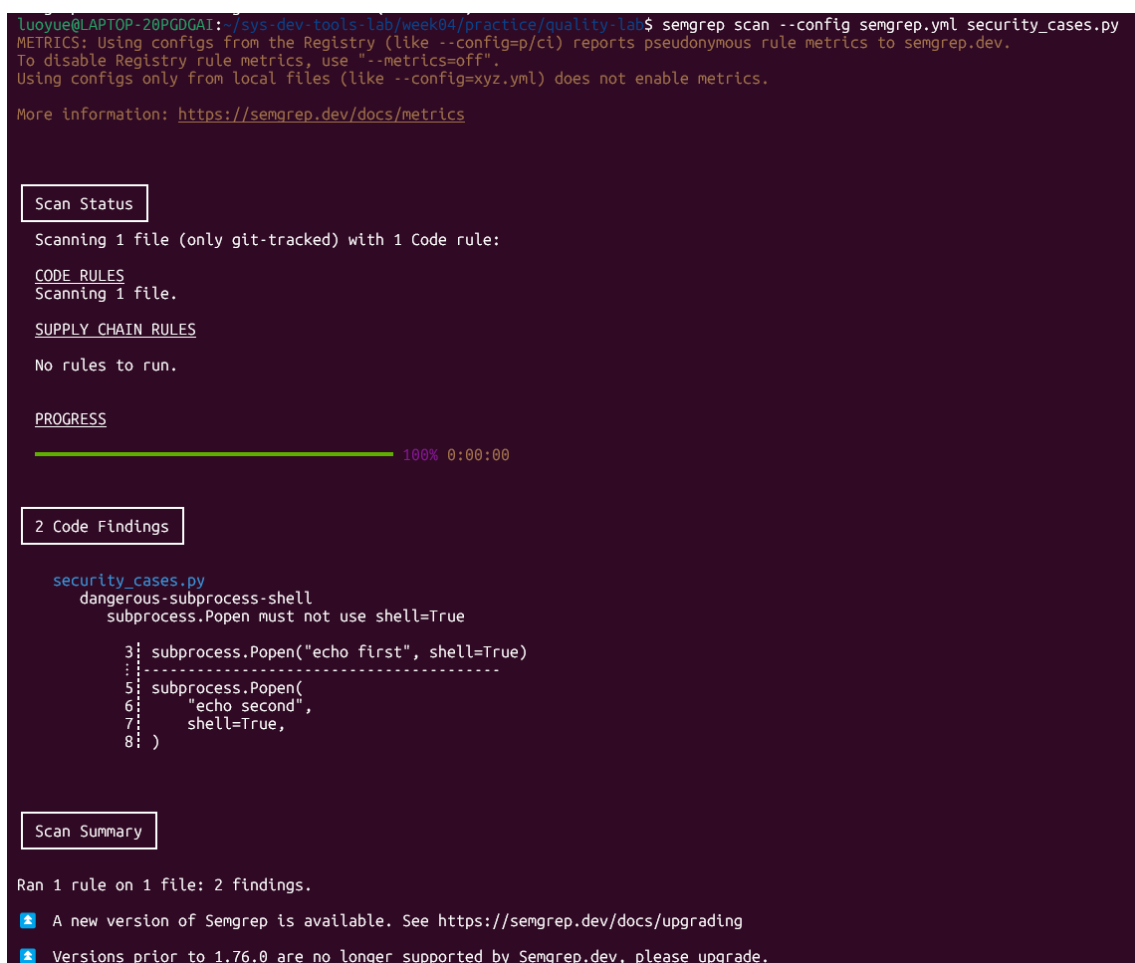
题目原文

写一个正则表达式，使用 `grep` 查找代码中 `subprocess.Popen(..., shell=True)` 的出现位置，再破坏正则模式，比较 `grep` 与 `Semgrep` 的匹配能力。同时在 Markdown 文档中使用正则查找替换，将项目符号 `-` 替换为 `*`，但不能误改其他用途的短横线。

实验步骤与分析

先在测试文件中写入单行和跨行的危险调用，使用 `grep -nE` 检查简单模式，再使用 `Semgrep` 规则进行结构化匹配。随后使用 `^(\\s*)-\\s+` 查找行首项目符号，并将捕获的缩进保留后替换为星号。该限制条件可以避免误改普通文本中的连字符。

实验结果与解释



```
luoyue@LAPTOP-20PGDGA1: ~/sys-dev-tools-lab/week04/practice/quality-lab$ semgrep scan --config semgrep.yml security_cases.py
METRICS: Using configs from the Registry (like --config=p/ci) reports pseudonymous rule metrics to semgrep.dev.
To disable Registry rule metrics, use "--metrics=off".
Using configs only from local files (like --config=xyz.yml) does not enable metrics.
More information: https://semgrep.dev/docs/metrics

Scan Status
Scanning 1 file (only git-tracked) with 1 Code rule:

CODE RULES
Scanning 1 file.

SUPPLY CHAIN RULES
No rules to run.

PROGRESS
100% 0:00:00

2 Code Findings

security_cases.py
dangerous-subprocess-shell
subprocess.Popen must not use shell=True

3 | subprocess.Popen("echo first", shell=True)
  | .....
5 | subprocess.Popen(
6 |     "echo second",
7 |     shell=True,
8 | )

Scan Summary
Ran 1 rule on 1 file: 2 findings.

A new version of Semgrep is available. See https://semgrep.dev/docs/upgrading
Versions prior to 1.76.0 are no longer supported by Semgrep.dev, please upgrade.
```

图 27: 课后练习九: `grep` 与 `Semgrep` 检查危险调用

```

119: (??) matches the group of a pattern
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ grep -nE '^[[:space:]]*-[[:space:]]+' code-quality.md | head
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ grep -nE '^[[:space:]]*-[[:space:]]+' code-quality.md | head
16:* [Formatting](#formatting)
17:* [Linting](#linting)
18:* [Testing](#testing)
19:* [Pre-commit hooks](#pre-commit-hooks)
20:* [Continuous integration](#continuous-integration)
21:* [Command runners](#command-runners)
101:* `abc` --- matches the literal "abc".
102:* `missing|semester` --- matches the string "missing" or the string "semester".
103:* `\d{4}-\d{2}-\d{2}` --- matches dates in YYYY-MM-DD format, such as "2026-01-14". Beyond ensuring that the string consists of four digits, a dash, two digits, a dash, and two digits, this does not validate the date, so "2026-01-99" matches this regex pattern too.
104:* `.+@.+` --- matches email addresses, strings that contain some text, then an "@", and then some more text. This does only the most basic validation and matches strings like "nonsense@@email". A regex that matches email addresses with no false positives or negatives [exists](https://pdw.ex-parrot.com/Mail-RFC822-Address.html) but is impractical.
luoyue@LAPTOP-20PGDGA1:~/sys-dev-tools-lab/week04/practice/quality-lab$ diff -u /tmp/code-quality.before.md code-quality.md | head -80
--- /tmp/code-quality.before.md 2026-09-21 16:18:07.649055738 +0800
+++ code-quality.md 2026-09-21 17:17:17.879381297 +0800
@@ -13,12 +13,12 @@

There are a variety of tools and techniques that support developers in writing high-quality code. In this lecture, we'll cover:

-- [Formatting](#formatting)
-- [Linting](#linting)
-- [Testing](#testing)
-- [Pre-commit hooks](#pre-commit-hooks)
-- [Continuous integration](#continuous-integration)

```

图 28: 课后练习九: Markdown 项目符号正则替换

grep 能匹配同一行的危险调用，但对跨行写法不够稳定；Sengrep 根据语法结构识别危险调用。Markdown 替换只处理行首项目符号，保留了普通文本中的短横线。

3.10 JSON 字段提取与解析器

题目原文

写一个正则表达式，从形如 `{"name": "Alyssa P. Hacker", "college": "MIT"}` 的 JSON 结构中捕获 name。处理 name 中包含转义双引号的情况，并使用 JSON 解析器从标准输入读取数据并输出 name。

实验步骤与分析

首先使用带有非贪婪和转义处理的正则提取 name 字段：

```
"name"\s*:\s*"((?:\\.|[^\"])*)"
```

该模式允许字段值中出现经过 JSON 转义的双引号。随后使用 Python 的 json 模块读取标准输入，并通过 json.load 访问 name 字段。解析器能够遵守完整 JSON 语法，适合实际实现。

实验结果与解释

```
luoyue@LAPTOP-20PGDGAI:~$ cd ~/sys-dev-tools-lab/week04/practice
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice$ mkdir -p json-regex
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice$ cd json-regex
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice/json-regex$ nano regex_demo.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice/json-regex$ python3 regex_demo.py
Alyssa P. Hacker
Alyssa "Ace" Hacker
```

图 29: 课后练习十: 正则提取 JSON 中的 name 字段

```
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice/json-regex$ nano json_name.py
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice/json-regex$ printf '%s\n' \
> '{"name": "Alyssa P. Hacker", "college": "MIT"}' |
> python3 json_name.py
Alyssa P. Hacker
luoyue@LAPTOP-20PGDGAI:~/sys-dev-tools-lab/week04/practice/json-regex$ printf '%s\n' \
> '{"name": "Alyssa \"Ace\" Hacker", "college": "MIT"}' |
> python3 json_name.py
Alyssa "Ace" Hacker
```

图 30: 课后练习十: 使用 Python JSON 解析器输出 name

正则方法能够提取普通姓名并处理转义双引号；JSON 解析器输出了相同字段值。比较两种方法可以看出，正则适合演示字符串模式匹配，而实际处理结构化 JSON 时应优先使用专用解析器。

4 实验总结与感悟

本周实验围绕代码质量、增量构建、数据处理、软件交付和自动化协作展开。第 13 题通过 `pytest`、`ruff` 和 `check.sh` 建立了可重复执行的本地质量门禁，使代码格式、静态检查和功能测试能够由统一命令完成。第 14 题通过 `Makefile` 声明文件之间的依赖关系，验证了 `Make` 能够根据源文件变化只重建必要产物，从而提高构建效率。

第 15 题将 JSON 文件通过本地 HTTP 服务提供，再结合 `curl` 与 `jq` 完成数据获取、筛选、排序和 Markdown 报告生成，体现了命令行工具在数据处理任务中的组合能力。第 16 题则将错误复现、测试失败、代码修复、质量验证、构建 `wheel` 和计算 SHA-256 校验值串联起来，形成了较完整的软件交付流程。

课后练习进一步补充了 `pre-commit`、覆盖率、持续集成、`Semgrep` 和 JSON 解析等方法。这些练习使我认识到，质量控制应当贯穿开发、测试、提交和交付的全过程；AI agent 可以参与格式修复和测试设计，但其结果仍需通过命令执行、覆盖率数据和人工审查进行验证。代码不仅应当能够运行，还应当具备明确的测试依据、可重复的构建过程和可验证的交付产物。